MSM Homework 03

Lou Yunuo, 22471297

2025 年 4 月 11 日

1 Q1

1.1 Generating data

We use the function to generate data as our sample set that fits criteria:

$$X_i \sim N(0,1) \quad \epsilon_i \sim N(0, 0.5^2)$$

$$Y_i = X_i + \sqrt{|X_i|} \cdot \epsilon_i$$

The code for this function is shown below:

```
1 def generate_data(n):
2     np.random.seed(42)
3     X = np.random.normal(loc=0, scale=1, size=n)
4     epsilon  = np.random.normal(loc=0, scale=0.5, size=n)
5     Y = X + np.sqrt(abs(X)) * epsilon
6     data = pd.DataFrame({'X': X, 'epsilon': epsilon,  'Y': Y})
7     return data
```

Data generated as below:

	X	epsilon	Y
0	0.496714	-0.414498	0.204585
1	-0.138264	-0.280091	-0.242413
2	0.647689	0.373647	0.948396
3	1.523030	0.305185	1.899662
4	-0.234153	-0.010451	-0.239210

图 1: Preview: Generated Sample Data

1.2 Q(a): Naive Bootstrap

Since our sample set as a whole has a relatively small number of data strips, we can do the OLS regression with 300 strips of data sampled at a time, which will also make the estimates from each bootstrap closer to the whole. When the sample set itself is large, or when computational resources are very limited, it is also feasible to choose any amount of data less than or equal to the sample set size n for estimation each time.

In this experiment, we resampled 300 pieces of data at a time for a total of 1,000 bootstrap estimations, and the variance of the final β estimates obtained is:

- Variance of Intercept: 0.0006415730430090519
- Variance of Coefficient: 0.0014760761719530544

The code is shown below:

```
1  import numpy as np
2  from sklearn.linear_model import LinearRegression
3  import generated_data as gd
4
5  def fit_ols(x, y):
6
7      model = LinearRegression()
8      model.fit(x.reshape(-1, 1), y)
9
10     beta_ols_intercept = model.intercept_
11     beta_ols_coef = model.coef_
12
13     return beta_ols_intercept, beta_ols_coef
14
15 def naive_bootstrap(boot_size, b_times, x, y):
16
17     bootstrap_betas_intercept = []
18     bootstrap_betas_coef = []
```

```
19
20      for _ in range(b_times):
21          indices = np.random.randint(0, y.shape[0], boot_size)
22          X_boot = x[indices]
23          Y_boot = y[indices]
24
25          intercept_boot, coef_boot = fit_ols(X_boot, Y_boot)
26          bootstrap_betas_intercept.append(intercept_boot)
27          bootstrap_betas_coef.append(coef_boot)
28
29      bootstrap_betas_intercept = np.array(bootstrap_betas_intercept)
30      bootstrap_betas_coef = np.array(bootstrap_betas_coef)
31      var_intercept = np.var(bootstrap_betas_intercept, ddof=1)
32      var_slope = np.var(bootstrap_betas_coef, ddof=1)
33      print(f"var_intercept:␣{var_intercept}")
34      print(f"var_slope:␣{var_slope}")
35
36      return var_intercept, var_slope
37
38  data_300 = gd.generate_data(300)
39  xi = np.array(data_300['X'])
40  yi = np.array(data_300['Y'])
41
42  naive_bootstrap(300, 1000, xi, yi)
```

1.3 Q(b): Wild Bootstrap

wild bootstrap asks us to create a new array of disturbance $\hat{\epsilon_i}$

$$\hat{\epsilon_i} = \epsilon_i \cdot w_i$$

$$P(w_i = -1) = P(w_i = 1) = 0.5$$

The variance of the β estimator obtained by this method is:

- Variance of Intercept: 0.0005926869525823164
- Variance of Coefficient: 0.001683562360822213

So we need to adjust *generate_data* function as below, *fit_ols* and *naive_bootstrap* functions remain as before:

```
1  def generate_data(n):
2
3      np.random.seed(42)
4
5      X = np.random.normal(loc=0, scale=1, size=n)
6      epsilon  = np.random.normal(loc=0, scale=0.5, size=n)
7      Y = X + np.sqrt(abs(X)) * epsilon
8      W = np.random.choice([1, -1], size=n)
9      epsilon_wild = epsilon * W
10     Y_wild = X + np.sqrt(abs(X)) * epsilon_wild
11
12     data = pd.DataFrame({'X': X
13                          , 'epsilon': epsilon
14                          , 'Y': Y
15                          , 'W': W
16                          , 'Y_wild': Y_wild})
17
18     return data
```

1.4 Q(c): Comment

1.4.1 Visualize the Distribution of β Estimates

We plot histograms of the estimators of the $\hat{\beta}_i$,including coefficient and intercept, calculated by the two methods, respectively, in order to observe their distributions and more intuitively compare the results obtained by the two methods.

The code is shown below:

```
def draw_histogram(data, content, method, color):
    plt.figure( figsize =(8, 6))
    plt.hist(data, bins=10, color=color, edgecolor='black', alpha=0.7)

    plt.title (f'Histogram␣of␣{content}␣using␣{method}', fontsize=16)
    plt.xlabel(f'{content}', fontsize=14)
    plt.ylabel(f'Frequency', fontsize=14)

    plt.grid(axis='y', linestyle ='--', alpha=0.7)
    plt.savefig(f'{method}-{content}.png', dpi=300, bbox_inches='
        tight')
```

The results of estimated **coefficient**:

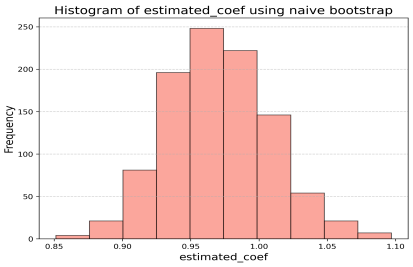

图 2: Naive Bootstrap

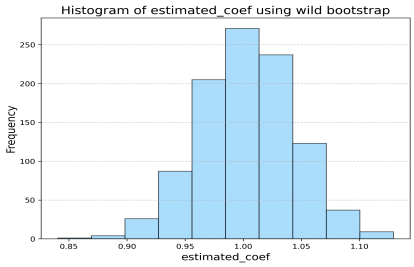

图 3: Wild Bootstrap

The results of estimated **intercept**:

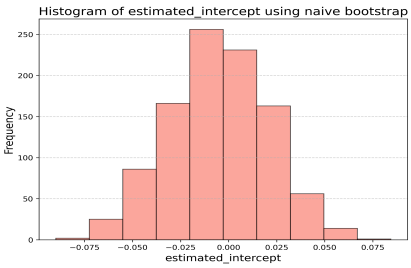

图 4: Naive Bootstrap

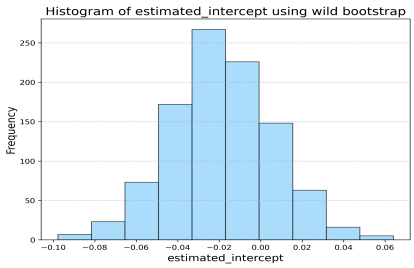

图 5: Wild Bootstrap

1.4.2 As for the results

It can be observed that there are slight differences between the two methods for estimating the variance of intercept and coefficient. Specifically, the intercept variance given by Wild Bootstrap is slightly smaller than the result of the naive self-help method, while the coefficient variance is slightly larger than the result of the naive self-help method.

1.4.3 As for the reasons

Wild Bootstrap is especially useful in cases of heteroscedasticity or non-normal error terms. In this case, since $Y_i = X_i + \sqrt{|X_i|} \cdot \epsilon_i$, the distribution of ϵ_i may result in a certain heteroscedasticity of the model error term. Therefore, Wild Bootstrap may more accurately reflect the true variability of the data.

The naive self-help method assumes that the residuals are independently and equally distributed, which is usually valid when dealing with simple linear regression problems, but if the data violates these assumptions (such as heteroscedasticity), its estimates may not be as accurate as Wild Bootstrap's.

2 Q2

2.1 Q(d): Estimate $\hat{\theta}$ in Lasso Model

We choose Lasso regression as the research object. Firstly, a simulated data set is constructed, which contains 300 sample data pieces and 20 feature variables, among which the theta corresponding to the first 5 feature variables is not 0. We built the dataset using the simplest linear model.

The code for this function is shown below:

```
1  def generate_data(n_samples, n_features):
2      np.random.seed(42)
3      X_array = np.random.randn(n_samples, n_features)
4      column_name = [f'X{i␣+␣1}' for i in range(n_features)]
5      X_df = pd.DataFrame(X_array, columns=column_name)
6      True_Theta = np.zeros(n_features)
7      True_Theta[:5] = np.array([1, 2, 3, 4, 5])
8      Epsilon = np.random.normal(loc=0, scale=0.5, size=n_samples)
9      Y_array = X_array.dot(True_Theta) + Epsilon
10     Y_df = pd.DataFrame(Y_array, columns=['Y'])
11     Data = pd.concat([X_df, Y_df], axis=1)
12     return Data, X_array, X_df, True_Theta, Epsilon, Y_array, Y_df
```

Data generated as below:

	X1	X2	X3	...	X19	X20	Y
0	0.496714	-0.138264	0.647689	...	-0.908024	-1.412304	6.527563
1	1.465649	-0.225776	0.067528	...	-1.328186	0.196861	-7.519691
2	0.738467	0.171368	-0.115648	...	0.331263	0.975545	-8.333797
3	-0.479174	-0.185659	-1.106335	...	0.091761	-1.987569	-5.165692
4	-0.219672	0.357113	1.477894	...	0.005113	-0.234587	-1.294389

图 6: Preview: Generated Sample Data

We then build a Lasso model to fit the data to get the original estimates $\hat{\theta}_i$ and provide a wrapped functional method for the bootstrap estimation method. It is worth noting that the Lasso model needs to use cross-validation method to determine the most parameters during the construction process, so we have improved the relevant code accordingly.

The code for this function is shown below:

```
def fit_lasso(x_train, y_train):
    lasso_cv = LassoCV(cv=5, random_state=42).fit(x_train, y_train)
    optimal_alpha = lasso_cv.alpha_
    lasso_optimal = Lasso(alpha=optimal_alpha, fit_intercept=True).fit
        (x_train, y_train)
    theta_hat = lasso_optimal.coef_
    return theta_hat, optimal_alpha
```

We first used the complete sample set data for estimation, and got the original estimates $\hat{\theta}_i$ as follows:

```
theta_hat_original_l: [ 9.37357130e-01  1.94974721e+00  2.93811016e+00  3.92577548e+00
  5.03353221e+00 -5.24988287e-03  0.00000000e+00  0.00000000e+00
  4.22923487e-04  0.00000000e+00  0.00000000e+00 -0.00000000e+00
  0.00000000e+00  3.67108906e-02  0.00000000e+00  0.00000000e+00
 -2.37909305e-02  0.00000000e+00 -0.00000000e+00 -0.00000000e+00]
```

图 7: The Original Estimates: $\hat{\theta}_i$

Then, we applied Lasso regression and Bootstrap method to estimate the deviation, taking the number of resampling for each time as 100, and conducted $B = 1000$ resampling estimates in total.

The code for this function is shown below:

```
def bootstrap(boot_size, b_times, x_train, y_train, theta_hat_original)
    :
    bootstrap_thetas = []

    for _ in range(b_times):
        indices  = np.random.randint(0, y_train.shape[0], boot_size)
        x_boot = x_train[indices]
        y_boot = y_train[indices]

        theta_hat_boot, alpha_selected = fit_lasso(x_boot, y_boot)
        bootstrap_thetas.append(theta_hat_boot)

```

```
    bootstrap_thetas = np.array(bootstrap_thetas)
    bias = np.mean(bootstrap_thetas, axis=0) - theta_hat_original

    return bias
```

The end result is as follows:

```
bias: [-0.00759853 -0.01394076 -0.00782709 -0.00869482 -0.00753179 -0.01293407
  0.00819511  0.00355923  0.01821735  0.00262841  0.00207468 -0.00378828
  0.01227259  0.00250413  0.00765684  0.00329124 -0.00636508  0.01428576
 -0.00997276 -0.00162939]
```

图 8: The Bias of $\hat{\theta}_i$

Theoretically, B is determined by sample size, statistical stability, computing resources, confidence level requirements, etc. Generally speaking, when the sample size is large, the statistical stability is strong, the computing resources are small, and the confidence level is not high, the number of B can be relatively small, and the other way around, it needs more.

2.2 Q(e): Finding *C.I.* Using Percentile Method

Find a 95% confidence interval for θ_0 using the percentile method.
The code for this function is shown below:

```
1  def ci_percentile(bootstrap_thetas, alpha_1):
2      CI = pd.DataFrame(columns=['feature', 'lower_bound', '
           upper_bound'])
3
4      feature_index = [f'X{i␣+␣1}' for i in range(bootstrap_thetas.shape
           [1])]
5      lower_bound = np.percentile(bootstrap_thetas, alpha_1 / 2 * 100,
           axis=0)
6      upper_bound = np.percentile(bootstrap_thetas, (1 - alpha_1 / 2) *
            100, axis=0)
7
8      CI['feature'] = feature_index
9      CI['lower_bound'] = lower_bound
10     CI['upper_bound'] = upper_bound
11
12     return CI
```

2.3 Q(f): Finding *C.I.* Using Percentile-t Method

Find a 95% confidence interval for θ_0 using the percentile-t method.
The code for this function is shown below:

```
1  def ci_percentile_t(bootstrap_thetas, theta_hat_original, alpha_2):
2      CI = pd.DataFrame(columns=['feature', 'lower_bound', '
           upper_bound'])
3      feature_index = [f'X{i␣+␣1}' for i in range(bootstrap_thetas.shape
           [1])]
4
5      t_stats = (bootstrap_thetas - theta_hat_original) / np.std(
           bootstrap_thetas, axis=0, ddof=1)
```

```
    lower_t = np.percentile(t_stats, alpha_2 / 2 * 100, axis=0)
    upper_t = np.percentile(t_stats, (1 - alpha_2 / 2) * 100, axis=0)

    ci_lower_t = theta_hat_original + lower_t * np.std(
        bootstrap_thetas, axis=0, ddof=1)
    ci_upper_t = theta_hat_original + upper_t * np.std(
        bootstrap_thetas, axis=0, ddof=1)

    CI['feature'] = feature_index
    CI['lower_bound'] = ci_lower_t
    CI['upper_bound'] = ci_upper_t

    return CI
```

The results of 95% CI for θ_o using 2 methods are as follows:

	feature	lower_bound	upper_bound
0	X1	0.816192	1.043256
1	X2	1.829401	2.056606
2	X3	2.834705	3.028078
3	X4	3.814701	4.011341
4	X5	4.923729	5.119523
5	X6	-0.100755	0.040163
6	X7	-0.056136	0.095617
7	X8	-0.048769	0.063291
8	X9	-0.023611	0.120344
9	X10	-0.062587	0.064150
10	X11	-0.073222	0.065725
11	X12	-0.074488	0.060815
12	X13	-0.024231	0.084420
13	X14	0.000000	0.129496
14	X15	-0.046049	0.080786
15	X16	-0.048430	0.069230
16	X17	-0.135284	0.000038
17	X18	-0.041988	0.101958
18	X19	-0.093494	0.046157
19	X20	-0.081963	0.063566

图 9: percentile method

	feature	lower_bound	upper_bound
0	X1	0.816192	1.043256
1	X2	1.829401	2.056606
2	X3	2.834705	3.028078
3	X4	3.814701	4.011341
4	X5	4.923729	5.119523
5	X6	-0.100755	0.040163
6	X7	-0.056136	0.095617
7	X8	-0.048769	0.063291
8	X9	-0.023611	0.120344
9	X10	-0.062587	0.064150
10	X11	-0.073222	0.065725
11	X12	-0.074488	0.060815
12	X13	-0.024231	0.084420
13	X14	0.000000	0.129496
14	X15	-0.046049	0.080786
15	X16	-0.048430	0.069230
16	X17	-0.135284	0.000038
17	X18	-0.041988	0.101958
18	X19	-0.093494	0.046157
19	X20	-0.081963	0.063566

图 10: percentile-t method

2.4 Q(g): Comment

2.4.1 Theoretically

The percentile method determines the confidence interval directly based on the distribution of the self-help sample estimates, while the percentile-t rule takes into account the standard error of the estimator, so it may give a more accurate interval estimate in some cases. Specifically, if the data does not satisfy the normality assumption or there is heteroscedasticity, the percentile-t method may be more appropriate.

2.4.2 Practically

From the experimental results, by comparing the confidence intervals obtained by percentile method and percentile-t method, we can observe that the results of both are almost identical. There are two possible reasons for this:

a) Symmetry of data distribution. If the Bootstrap distribution of the estimator (Lasso's regression coefficient $\hat{\theta}$) is symmetric, then the percentile method and the percentile-t method will give nearly identical confidence intervals. This is because in both methods, the construction of the confidence interval depends on the distribution shape of the estimators. For symmetric distributions, both methods tend to produce similar results.

b) Consistency of standard error. In the percentile-t method, we usually use the t statistic to adjust the confidence interval. If the standard errors of all Bootstrap samples are very close, then the effect of the t statistic may be cancelled out, resulting in results similar to using the percentile method directly.

3 Q3

3.1 Q(h): 75% sample quantile histogram

First, we need to generate a sample of size 1000 from a uniform distribution over (0, 1).

```
def generate_data(n):
    np.random.seed(42)
    data = np.random.uniform(0, 1, n)
    return data
```

Then, We will use the Bootstrap method to resample the data and calculate the 75% quantile of each resampling.

```
def bootstrap(b_times, n, data):
    bootstrap_quantiles = []
    for _ in range(b_times):
        indices  = np.random.randint(0, n, n)
        sample = data[indices]
        quantile_75 = np.percentile(sample, 75)
        bootstrap_quantiles.append(quantile_75)
    return bootstrap_quantiles
```

Finally we plot the distribution histogram of these quantiles.

```
def draw_histogram(array, color):
    plt.hist(array,  bins=30, alpha=0.7, color=color)
    plt.title('Bootstrap Distribution of the 75% Sample Quantile')
    plt.xlabel('75% Sample Quantile')
    plt.ylabel('Frequency')
    plt.savefig(f'3-h.png', dpi=300, bbox_inches='tight')
```

Here's the result:

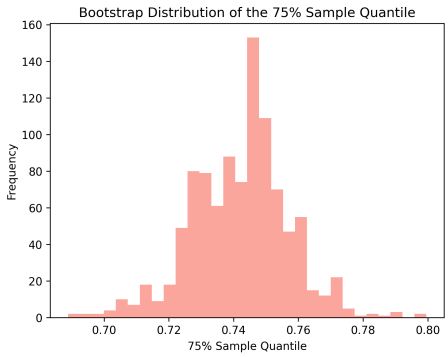

图 11: The Distribution of the 75% Sample Quantile

3.2 Q(i): How Large should B be?

In theory, the larger the B, the more accurate the estimate of the sample quantile distribution will be. Since there is no clear standard for the value of B, we try to take different values of B and plot the distribution of the 75% sample quantile, observe when B to what value, the distribution shape is basically stable. The code for this function is shown below:

```
1  def draw_histogram_try_b(ax, array, color, title):
2      ax.hist(array, bins=30, alpha=0.7, color=color)
3      ax.set_title(title, fontsize=12)
4      ax.set_xlabel('75% Sample Quantile', fontsize=10)
5      ax.set_ylabel('Frequency', fontsize=10)
6
7  def try_b(try_times, result, times_list, color):
8
9      fig, axes = plt.subplots(nrows=int(np.sqrt(try_times))
10                              , ncols=int(np.sqrt(try_times))
11                              , figsize=(12, 10)
12                              , sharex=True
13                              , sharey=True)
```

```
14
15      colors = [color] * len(result)
16      for i, ax in enumerate(axes.flat): # 遍历所有子图（axes.flat 展平
            二维数组）
17          draw_histogram_try_b(ax, result[i], colors[i], f'B␣=␣{
                times_list[i]}')
18
19      plt.tight_layout()
20      plt.savefig(f'3-i.png', dpi=300, bbox_inches=None)
21
22  n = 1000
23  data_3 = generate_data(n)
24  B_try_times = 25
25  q = 75
26  boot = 300
27   result_list = []
28  B_list = []
29  var_list = []
30  for b in range(B_try_times):
31      B = 20 + b * 20
32      B_list.append(B)
33      res = bootstrap(B, boot, n, data_3, q)
34       result_list.append(res)
35      var = round(np.sum(np.abs(np.array(res) - q/100) ** 2)/B, 6)
36      var_list.append(var)
37  try_b(B_try_times, result_list, B_list, var_list, 'salmon')
```

Here's the result, it's seen like With the increase of B, the estimated distribution results will be more stable, and relatively reliable when $B \geq 700$:

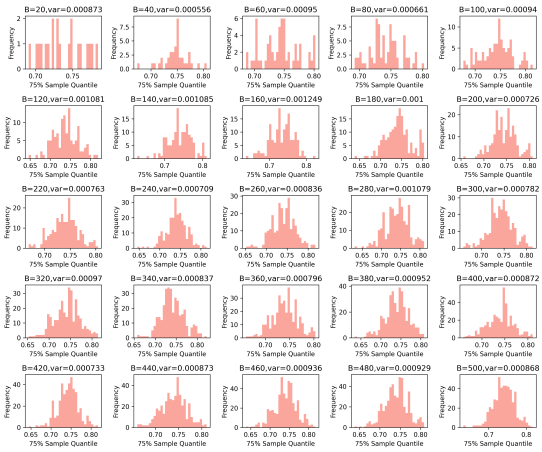

图 12: The Distribution changing with $B \in [20, 500]$

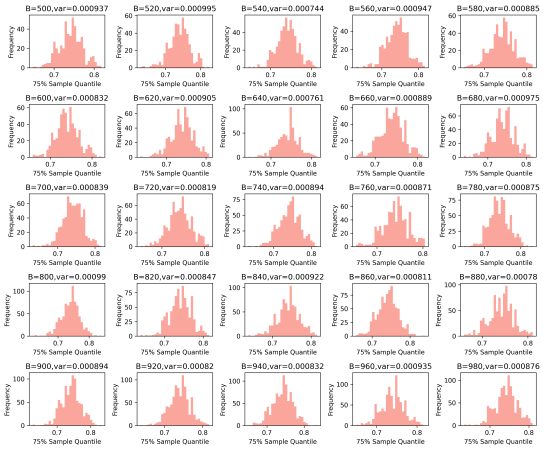

图 13: The Distribution changing with $B \in [520, 980]$

3.3 Q(j): 99.5% sample quantile histogram

To investigate this problem, we repeat the process in Q(h), but this time calculate the 99.5% quantile.

The result is as below:

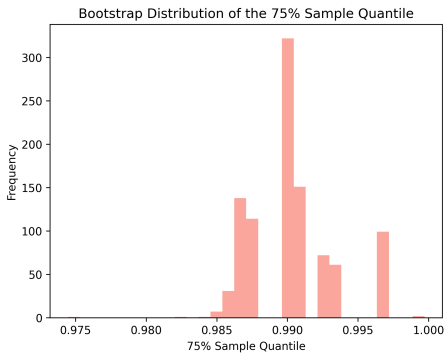

图 14: The Distribution of the 99.5% Sample Quantile

Since 75% quantile is in the middle region of the distribution, Bootstrap method can capture its distribution characteristics well. However, since the 99.5% quantile is located at the extreme end of the distribution, Bootstrap resampling may not provide enough information to accurately estimate this quantile, resulting in increased uncertainty in the estimate.

A Appendix of Full Code

A.1 1.py

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

def generate_data(n):
    """
    按照题目要求的分布，生成样本数据集
    :param n: 生成数据的条数
    :return: 数据集[DataFrame]
    """
    np.random.seed(42)

    X = np.random.normal(loc=0, scale=1, size=n)
    epsilon = np.random.normal(loc=0, scale=0.5, size=n)
    Y = X + np.sqrt(abs(X)) * epsilon
    W = np.random.choice([1, -1], size=n)
    epsilon_wild = epsilon * W
    Y_wild = X + np.sqrt(abs(X)) * epsilon_wild

    data = pd.DataFrame({'X': X
                         , 'epsilon': epsilon
                         , 'Y': Y
                         , 'W': W
                         , 'Y_wild': Y_wild})
    # print(data.head(8))

    return data

def fit_ols(x, y):
```

```
31      """
32      对X, Y训练数据进行OLS回归，得到参数 的估计量
33      :param x: 特征变量[Array]
34      :param y: 目标变量[Array]
35      :return:
36          斜率[array]
37          截距[Float64]
38      """
39      model = LinearRegression()
40      model.fit(x.reshape(−1, 1), y)
41
42      beta_ols_intercept = model.intercept_
43      beta_ols_coef = model.coef_
44
45      return beta_ols_intercept, beta_ols_coef
46
47
48  def draw_histogram(data, content, method, color):
49      """
50      对结果绘制直方图，以便观察分布情况
51      :param data: 画图的数据[array]
52      :param content: 是对哪个数据画的图[string]
53      :param method: 是用那种方法得到的结果[string]
54      :param color: 直方图的填充颜色[string]
55      :return: 将绘制的图片以.png格式保存在本地路径
56      """
57      plt.figure( figsize =(8, 6))
58      plt.hist(data, bins=10, color=color, edgecolor='black', alpha=0.7)
59
60      plt.title (f'Histogram␣of␣{content}␣using␣{method}', fontsize=16)
61      plt.xlabel(f'{content}', fontsize=14)
62      plt.ylabel(f'Frequency', fontsize=14)
63
```

```
64      plt.grid(axis='y',  linestyle ='--', alpha=0.7)
65      plt.savefig(f'{method}-{content}.png', dpi=300, bbox_inches='
            tight')
66
67
68  def naive_bootstrap(boot_size, b_times, x, y):
69      """
70      使用naive_bootstrap，对样本进行OLS回归，计算参数的方差
71      :param boot_size: 每次bootstrap抽取的样本数（样本总数）[int]
72      :param b_times: bootstrap的次数[int]
73      :param x: 特征变量[Array]
74      :param y: 目标变量[Array]
75      :return:
76          截距的方差[Float64]
77          斜率的方差[Float64]
78      """
79      bootstrap_betas_intercept = []
80      bootstrap_betas_coef = []
81
82      for _ in range(b_times):
83          indices  = np.random.randint(0, y.shape[0], boot_size)
84          X_boot = x[indices]
85          Y_boot = y[indices]
86
87          intercept_boot, coef_boot = fit_ols(X_boot, Y_boot)
88          bootstrap_betas_intercept.append(intercept_boot)
89          bootstrap_betas_coef.append(coef_boot)
90
91      bootstrap_betas_intercept = np.array(bootstrap_betas_intercept)
92      bootstrap_betas_coef = np.array(bootstrap_betas_coef)
93      var_intercept = np.var(bootstrap_betas_intercept, ddof=1)
94      var_slope = np.var(bootstrap_betas_coef, ddof=1)
95      print(f"var_intercept:␣{var_intercept}")
```

```
96      print(f"var_slope:␣{var_slope}")
97
98      return var_intercept, var_slope, bootstrap_betas_intercept,
            bootstrap_betas_coef
99
100 # Q(a)
101 data_300 = generate_data(300)
102 xi = np.array(data_300['X'])
103 yi = np.array(data_300['Y'])
104
105 var_i, var_s, boot_i, boot_c = naive_bootstrap(300, 1000, xi, yi)
106
107 draw_histogram(boot_i, 'estimated_intercept', 'naive␣bootstrap', '
        salmon')
108 draw_histogram(boot_c, 'estimated_coef', 'naive␣bootstrap', 'salmon')
109
110 # Q(b)
111 data_300 = generate_data(300)
112 xi = np.array(data_300['X'])
113 yi = np.array(data_300['Y_wild'])
114
115 var_i_w, var_s_w, boot_i_w, boot_c_w = naive_bootstrap(300, 1000,
        xi, yi)
116
117 draw_histogram(boot_i_w, 'estimated_intercept', 'wild␣bootstrap', '
        lightskyblue')
118 draw_histogram(boot_c_w, 'estimated_coef', 'wild␣bootstrap', '
        lightskyblue')
```

A.2 2.py

```
1  import numpy as np
2  import pandas as pd
3  from sklearn.linear_model import Lasso, LassoCV
4  from sklearn.model_selection import train_test_split
5
6  def generate_data(n_samples, n_features):
7      """
8      生成一个适用于lasso回归的数据集
9      :param n_samples: 样本容量[int]
10     :param n_features: 特征变量数量[int]
11     :return: 数据集[dataframe, array]
12     """
13     np.random.seed(42)
14
15     X_array = np.random.randn(n_samples, n_features)
16     column_name = [f'X{i␣+␣1}' for i in range(n_features)]
17     X_df = pd.DataFrame(X_array, columns=column_name)
18     True_Theta = np.zeros(n_features)
19     True_Theta[:5] = np.array([1, 2, 3, 4, 5])
20     Epsilon = np.random.normal(loc=0, scale=0.5, size=n_samples)
21     Y_array = X_array.dot(True_Theta) + Epsilon
22     Y_df = pd.DataFrame(Y_array, columns=['Y'])
23     Data = pd.concat([X_df, Y_df], axis=1)
24     # print(Data.head(5))
25
26     return Data, X_array, X_df, True_Theta, Epsilon, Y_array, Y_df
27
28 def fit_lasso(x_train, y_train):
29     """
30     构建lasso模型拟合函数，首先使用CV获取最佳超参数，再将最优参数
           输入拟合模型
31     :param x_train: 训练集−特征[array]
```

```
    :param y_train: 训练集−目标[array]（压平到一维）
    :return:
        theta_hat: 估计量 [array]
        optimal_alpha: lasso模型的最优参数 [array]
    """
    lasso_cv = LassoCV(cv=5, random_state=42).fit(x_train, y_train)
    optimal_alpha = lasso_cv.alpha_
    # print(f"optimal_alpha: {optimal_alpha}")

    lasso_optimal = Lasso(alpha=optimal_alpha, fit_intercept=True).fit
        (x_train, y_train)
    theta_hat = lasso_optimal.coef_
    # print(f"theta_hat: {theta_hat}")

    return theta_hat, optimal_alpha

def bootstrap(boot_size, b_times, x_train, y_train, theta_hat_original)
    :
    """
    bootstrap估计参数
    :param boot_size: 每次抽样的数量[int](  n_samples)
    :param b_times: 重抽样的次数[int]
    :param x_train: 训练集−特征[array]
    :param y_train: 训练集−目标[array]（压平到一维）
    :param theta_hat_original: 先用样本集全体拟合一次lasso，获取初始
        估计量[array]
    :return:
        bias：bootstrap得到的 估计值的均值相较于初始估计量的偏误[
            array]
        bootstrap_thetas：bootstrap得到的 估计值[array]
    """
    bootstrap_thetas = []

```

```
61      for _ in range(b_times):
62          indices = np.random.randint(0, y_train.shape[0], boot_size)
63          x_boot = x_train[indices]
64          y_boot = y_train[indices]
65
66          theta_hat_boot, alpha_selected = fit_lasso(x_boot, y_boot)
67          bootstrap_thetas.append(theta_hat_boot)
68
69      bootstrap_thetas = np.array(bootstrap_thetas)
70      bias = np.mean(bootstrap_thetas, axis=0) - theta_hat_original
71
72      # print(f"bias: {bias}")
73
74      return bias, bootstrap_thetas
75
76  def ci_percentile(bootstrap_thetas, alpha_1):
77      """
78      用 percentile方法求估计量 在一定水平下的置信区间
79      :param bootstrap_thetas: bootstrap得到的 估计值[array]
80      :param alpha_1: 置信水平[float64]
81      :return: 各特征变量的参数 的置信区间[dataframe]
82      """
83      CI = pd.DataFrame(columns=['feature', 'lower_bound', '
            upper_bound'])
84
85      feature_index = [f'X{i␣+␣1}' for i in range(bootstrap_thetas.shape
            [1])]
86      lower_bound = np.percentile(bootstrap_thetas, alpha_1 / 2 * 100,
            axis=0)
87      upper_bound = np.percentile(bootstrap_thetas, (1 - alpha_1 / 2) *
             100, axis=0)
88
89      CI['feature'] = feature_index
```

```
90      CI['lower_bound'] = lower_bound
91      CI['upper_bound'] = upper_bound
92      # print(CI)
93      return CI
94
95  def ci_percentile_t(bootstrap_thetas, theta_hat_original, alpha_2):
96      """
97      用 percentile-t方法求估计量 在一定水平下的置信区间
98      :param bootstrap_thetas: bootstrap得到的 估计值[array]
99      :param theta_hat_original: 初始估计量[array]
100     :param alpha_2: 置信水平[float64]
101     :return: 各特征变量的参数 的置信区间[dataframe]
102     """
103     CI = pd.DataFrame(columns=['feature', 'lower_bound', '
            upper_bound'])
104     feature_index = [f'X{i␣+␣1}' for i in range(bootstrap_thetas.shape
            [1])]
105
106     t_stats = (bootstrap_thetas - theta_hat_original) / np.std(
            bootstrap_thetas, axis=0, ddof=1)
107     lower_t = np.percentile(t_stats, alpha_2 / 2 * 100, axis=0)
108     upper_t = np.percentile(t_stats, (1 - alpha_2 / 2) * 100, axis=0)
109
110     ci_lower_t = theta_hat_original + lower_t * np.std(
            bootstrap_thetas, axis=0, ddof=1)
111     ci_upper_t = theta_hat_original + upper_t * np.std(
            bootstrap_thetas, axis=0, ddof=1)
112
113     CI['feature'] = feature_index
114     CI['lower_bound'] = ci_lower_t
115     CI['upper_bound'] = ci_upper_t
116     # print(CI)
117     return CI
```

```
118
119
120 data_300_20, x_array, x_df, theta, epsilon, y_array, y_df =
        generate_data(300, 20)
121 xi = np.array(x_df)
122 yi = np.array(y_df)
123 yi = np.squeeze(yi)
124 x_train_l, x_test_l, y_train_l, y_test_l = train_test_split(xi, yi,
        test_size=0.2, random_state=42)
125
126 theta_hat_original_l, alpha = fit_lasso(x_train_l, y_train_l)
127 # print(f"theta_hat_original_l: {theta_hat_original_l}")
128 bias, thetas = bootstrap(100, 1000, x_train_l, y_train_l,
        theta_hat_original_l)
129
130 # Q(e) & Q(f)
131 alpha = 0.05
132 print('percentile')
133 ci_percentile(thetas, alpha)
134 print('percentile-t')
135 ci_percentile_t(thetas, theta_hat_original_l, alpha)
```

A.3 3.py

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  def generate_data(n):
5      """
6      创造适用于分位数重抽样的样本数据集
7      :param n: 样本容量[int]
8      :return: 样本数据集[array]
9      """
10     np.random.seed(42)
11     data = np.random.uniform(0, 1, n)
12     # print(data)
13     return data
14
15 def bootstrap(b_times, boot_size, n, data, q_level):
16     """
17     bootstrap方法实现
18     :param b_times: 重抽样次数[int]
19     :param boot_size: 每次抽样的数量[int]
20     :param n: 样本容量[int]
21     :param data: 样本数据集[array]
22     :param q_level: 分位数水平(0,100)[float64]
23     :return: bootstrap重抽样估计值[array]
24     """
25     bootstrap_quantiles = []
26
27     for _ in range(b_times):
28         indices  = np.random.randint(0, n, boot_size)
29         sample = data[indices]
30         quantile_value = np.percentile(sample, q_level)
31         bootstrap_quantiles.append(quantile_value)
32
```

```
33      return bootstrap_quantiles
34
35  def draw_histogram(array, color):
36      """
37      绘制估计值分布的直方图
38      :param array: 估计值[array]
39      :param color: 直方图填充颜色[string]
40      :return: 将绘制的图片以.png格式保存在本地路径
41      """
42      plt.hist(array, bins=30, alpha=0.7, color=color)
43      plt.title('Bootstrap Distribution of the 75% Sample Quantile')
44      plt.xlabel('75% Sample Quantile')
45      plt.ylabel('Frequency')
46      plt.savefig(f'3-j.png', dpi=300, bbox_inches='tight')
47
48  def draw_histogram_try_b(ax, array, color, title):
49      """
50      遍历绘组图
51      :param ax: 画布绘图点
52      :param array: 估计值[array]
53      :param color: 直方图填充颜色[string]
54      :param title: 图片标题[string]
55      :return: 将绘制的图片以.png格式保存在本地路径
56      """
57      ax.hist(array, bins=30, alpha=0.7, color=color)
58      ax.set_title(title, fontsize=12)
59      ax.set_xlabel('75% Sample Quantile', fontsize=10)
60      ax.set_ylabel('Frequency', fontsize=10)
61
62  def try_b(try_times, result, times, var, color):
63      """
64      用于遍历B，观察估计值分布情况改变
65      :param try_times: 尝试次数（须是完全平方数）[int]
```

```
66      :param result: 估计值[list]
67      :param times: 估计次数[list]
68      :param var: 每次的估计总方差[list]
69      :param color: 直方图填充颜色[string]
70      :return: 将绘制的图片以.png格式保存在本地路径
71      """
72      fig, axes = plt.subplots(nrows=int(np.sqrt(try_times))
73                               , ncols=int(np.sqrt(try_times))
74                               , figsize=(12, 10))
75
76      colors = [color] * len(result)
77      for i, ax in enumerate(axes.flat): # 遍历所有子图（axes.flat 展平
            二维数组）
78          draw_histogram_try_b(ax, result[i], colors[i], f'B={times[i]},
                var={var[i]}')
79
80      plt.tight_layout()
81      plt.savefig(f'3-i-2.png', dpi=300, bbox_inches=None)
82
83
84  n = 1000
85  data_3 = generate_data(n)
86
87  # Q(h)
88  B = n
89  q = 75
90  quantile = bootstrap(B, n, n, data_3, q)
91  draw_histogram(quantile, 'salmon')
92
93  # Q(i)
94  B_try_times = 25
95  q = 75
96  boot = 300
```

```
result_list = []
B_list = []
var_list = []
for b in range(B_try_times):
    B = 500 + b * 20
    B_list.append(B)
    res = bootstrap(B, boot, n, data_3, q)
    result_list.append(res)
    var = round(np.sum(np.abs(np.array(res) - q/100) ** 2)/B, 6)
    # var = round(abs(np.mean(res - q/100, axis=0), 4)
    var_list.append(var)

try_b(B_try_times, result_list, B_list, var_list, 'salmon')

# Q(j)
B = n
q = 99.5
quantile = bootstrap(B, n, n, data_3, q)
draw_histogram(quantile, 'salmon')
```